

DS 2500: Intermediate Programming with Data

Fall 2025

Homework 1: Pixar Films & Ratings

Assigned: September 9, 2025

Deadline: September 23, 2025 at 11:59 PM ET

Total Points: 1,000 pts

Overview and Learning Objectives

In this homework, you will be analyzing real [Pixar](#) movie data to practice essential Python programming concepts while discovering insights about popular animated films.

This assignment is intended to review the following Python programming concepts:

- Use/manipulation of various data types including dictionaries, lists, strings, numbers, and tuples.
- Control structures (conditional statements, loops, parameters, returns) to implement basic functions (including lambda functions) provided by a specification.
- Basic data visualization.
- Basic text file processing.
- Documenting code.

Submission Instructions

- Submit the Python file containing your solutions to DS2500 Gradescope by **11:59 PM ET September 23, 2025**.
- Late submission is allowed until **October 7, 2025** with a 100 point penalty. Submissions will not be accepted after the late submission deadline.
- You may submit to Gradescope as many times as you want to check your score and improve your score.

Additional Instructions

- Make sure to download all necessary files from **Canvas**. You are given a **starter code file** for this assignment.
- All files you read and write must be in the same directory as your Python script. This means you only need to provide the filename (e.g., `open("data.csv", "r")`) without specifying the full path. Make sure to do this to avoid triggering autograder test failures.
- Make sure **not to change the file names or the instructed variable names** to avoid triggering autograder test failures.
- You may **NOT** use Pandas or NumPy for this assignment.
- You may use `print()` to test and understand your code.

Grading Policies

Homeworks are broken into two components:

1. **Accuracy (820 points).** You'll answer quantitative questions about the dataset on gradescope. These answers are auto-graded. Gradescope can be a little picky, so make sure you don't put extraneous characters or whitespace in your answers -- and double-check the "correct/incorrect" confirmation!
2. **Code Quality (180 points).** You'll submit your code as well, which we will review and grade based on its modularity, readability, and reusability. Details are in the [DS2500 Grading Guidelines](#).

Dataset

`pixar_films.csv` contains information about 27 Pixar films including release dates, runtimes, ratings, and critic scores from various sources.

Columns:

- `number`: Order of release in chronological order
- `film`: Name of film
- `release_date`: Date film premiered/released (YYYY-MM-DD format)
- `run_time`: Film length or runtime in minutes
- `film_rating`: Motion Picture Association (MPA) film rating (G, PG, PG-13, etc.)
- `rotten_tomatoes`: Score from the American review-aggregation website Rotten Tomatoes (0-100)
- `metacritic`: Score from Metacritic where scores are weighted average of reviews (0-100)

Problem 01 - Data Cleaning and Loading (50 points)

The first thing we need to do before any analysis is to read and load the `pixar_films.csv` file and do some pre-processing or cleaning! This problem contains multiple parts so make sure you do all of them.

Write a function called `load_and_clean_pixar_data(filename)` that:

1. Load the csv file using `csv.DictReader` to a list of dictionary objects. This is already in the starter code file. Each dictionary object should represent a row of data in the csv file.
2. Identify rows to remove and format data fields: You will need to check for any rows where `film` is an empty string and remove it from the list. Also, make sure that `run_time`, `rotten_tomatoes`, and `metacritic` field values are all float values instead of string by using `float()` to cast a string value to float. *If any of the numerical fields contain an empty string, set it to None.*

Key Function Requirements:

- Use a loop to check each row

- Use conditionals to identify missing values
- Define the function with proper variable scope
- Return exactly the specified tuple format in the correct order

Args:

- `filename` (string): Name of csv file to load

Returns:

Tuple containing the following:

- `clean_data_list`: list of dictionaries, where each dictionary represents one clean film record
- `original_count`: integer of original number of rows
- `removed_count`: integer of rows removed
- `final_count`: integer of final number of rows

Problem 02 - Rotten Tomatoes Score Analysis (80 points)

What is the minimum Rotten Tomatoes score for a Pixar movie? What about the maximum and average?

Write a function called `calculate_rt_score_statistics(data)` that analyzes the Rotten Tomatoes scores in the dataset.

Key Function Requirements:

- **Cannot use** `sort()`, `min()`, `max()`, `mean()`, or `sum()` functions. Use loops and conditionals instead to calculate the statistics.
- **Can use** `round()` function to round the average score to 1 decimal place. More details on this function can be found in: <https://docs.python.org/3/library/functions.html#round>
- Must handle `None` values appropriately

Args:

- `data` (list): List of dictionaries containing cleaned Pixar films data

Returns:

A dictionary containing the following keys and values:

- `min_score` (float): Minimum Rotten Tomatoes score
- `max_score` (float): Maximum Rotten Tomatoes score
- `avg_score` (float): Average Rotten Tomatoes score rounded to 1 decimal place

Problem 03 - Popular Release Month (80 points)

Is there a specific month in the year when the majority of Pixar films are released/premiered?

Write a function called `get_most_popular_release_month(data)` that finds which month has the most Pixar film releases.

Function Requirements:

- Cannot use `max()` or `sort()` functions
- Use the provided `num2month` dictionary in the code file to convert month numbers to month names which is the required format to return

Args:

- `data` (list): List of dictionaries containing cleaned Pixar films data

Returns:

A tuple containing:

- `most_popular_month` (str): The month with the highest number of films released (e.g., "January", "February", etc.)
- `total_films` (int): The total number of films released in that month

Problem 04 - Movie Runtime (150 points)

This problem has three parts (4a, 4b, 4c). Complete all three functions.

Problem 4(a) - Longest and Shortest Films (50 points)

What are the longest and shortest films?

Write a function called `get_longest_and_shortest_films(data)` that finds the longest and shortest films by runtime.

Function Requirements:

- Cannot use `max()`, `min()`, or `sort()` functions
- Must handle `None` values in runtime data

Args:

- `data` (list): List of dictionaries containing cleaned Pixar films data

Returns:

A dictionary with the following keys:

- **shortest_film** (str): The title of the shortest film
- **longest_film** (str): The title of the longest film

Problem 4(b) - Runtime Categories (50 points)

How many short, medium, and long-length movies are in the dataset?

Write a function called **get_runtime_category_counts(data)** that categorizes films by runtime length.

Categories:

- **short**: Films with runtime < 90 minutes
- **medium**: Films with runtime between 90 and 110 minutes (inclusive)
- **long**: Films with runtime > 110 minutes

Args:

- **data** (list): List of dictionaries containing cleaned Pixar films data

Returns:

A dictionary with runtime categories as keys and their counts as values

Problem 4(c) - Average Runtime by Rating (50 points)

What is the average runtime for a user-given MPA rating?

Write a function called **get_runtime_by_rating(data, rating)** that calculates the average runtime for films with a specific MPA rating.

Args:

- **data** (list): List of dictionaries containing Pixar films data
- **rating** (str): The film rating to filter by (options: 'G', 'PG', 'PG-13', 'R')

Returns:

- **float**: The average runtime of films with the specified rating, rounded to 1 decimal place
- **None**: If the rating is not valid or if there are no films with that rating

Problem 05 - Original Films vs Sequels (280 points)

Are original movies considered to be better or sequels? Let's calculate the average Rotten Tomatoes score for each set of films to find out!

This problem has two parts (5a, 5b). Complete all functions in order.

Problem 5(a) - Filter Films by Type (240 points)

When you take a look at the data (on excel, Google sheets, or via print), you will notice that not all sequels end with a '2' or a '3'. For example: "Finding Dory" is a sequel of "Finding Nemo" and "Monsters University" is a sequel of "Monsters, Inc."

What would be the best way to distinguish original films from sequels?

Write a function called `get_films_by_type(data, type_filter)` that separates original films from sequels.

Hint:

- **NOTE:** some sequel titles don't have a '2' or '3' at the end.
- The title of the film is what can help you distinguish the type of film. What pattern do you see when you look at the titles of originals and their sequels? We encourage you to create a helper function that processes the movie titles in a way that can help you find movies with prequels and sequels.
- We can also take advantage of the fact that our dataset is ordered by release date so looping through it in order should help in finding which movies are originals and sequels.

Args:

- `data` (list): List of dictionaries containing Pixar films data
- `type_filter` (str): 'original' or 'sequel' to specify which type of movies we want returned

Returns:

`list`: A list of dictionaries containing films of the specified type

Problem 5(b) - Compare Original vs Sequel Scores (40 points)

Now that we have a method that can filter to just an original set of movies or sequels, it is just a matter of a few lines of code to figure out if the average Rotten Tomatoes score is higher for original movies or sequel movies.

Write a function called `calculate_originals_and_sequels_rt_scores(data)` that compares the average Rotten Tomatoes scores between original films and sequels.

Function Requirements:

- Use the `get_films_by_type()` function from 5(b)
- Can use `round()` function to round to 1 decimal place

Returns:

A dictionary with the following keys:

- `original_avg_rt_score` (float): Average RT score for original films
- `sequel_avg_rt_score` (float): Average RT score for sequels

Problem 06 - Top Films by Composite Score (180 points)

Just using only one review score to determine how well a movie was received by critics and the public does not seem fair. Is there a way to create a composite score and use that to rank our movies? Composite scores involve combining different review scores.

Write a function called `filter_top_five_films(data)` that ranks films using a composite scoring system.

Composite Score Formula:

- Composite Score = (Rotten Tomatoes × 0.4) + (Metacritic × 0.6)

Function Requirements:

- **Must use** at least one lambda function
- **Highly suggest using** `map()` and/or `sorted()` functions with lambda but is *not required*. To find more details on how to use these Python built-in functions, refer to: <https://docs.python.org/3/library/functions.html>
- Handle `None` values by assigning a composite score of 0

Args:

- `data` (list): List of dictionaries containing Pixar films data

Returns:

list: A list of dictionaries containing the top five films, sorted in descending order by composite score. Each dictionary contains:

- `film` (str): The title of the film
- `composite_score` (float): The composite score of the film

Testing Your Code

Include a main section that tests all your functions:

```
if __name__ == "__main__":
    print("HW 01: PIXAR FILMS DATA ANALYSIS")

    # Test Problem 1
    data, original_count, removed_count, final_count =
load_and_clean_pixar_data('pixar_films.csv')
    print(f"Original Count: {original_count}, Removed Count:
{removed_count}, Final Count: {final_count}")

    # Test remaining problems...
```

Tips for Success

1. **Start Early:** This assignment covers many fundamental concepts - give yourself plenty of time
2. **Test Incrementally:** Test each function as you write it
3. **Handle Edge Cases:** Always check for **None** values and empty data
4. **Read Instructions Carefully:** Pay attention to which built-in functions you can and cannot use
5. **Use Helper Functions:** Break complex problems into smaller, manageable pieces if needed
6. **Comment Your Code:** Write clear comments explaining your logic
7. **30-minute Guideline:** If you get stuck on a homework problem, come by office hours or post on Piazza! We recommend you spend about 30 minutes trying to figure out a problem, and then ask for help. This will be enough time that you can try a few things to get unstuck, but not SO much time that you're banging your head against the wall. Try for 30 minutes, then ask us.

Hope you learned something interesting about Pixar films through this assignment!